Strings II

Tries, Aho-Corasick, and an introduction to suffix arrays

Learning goals

- Build a **trie** over a set of words and answer prefix queries
- Use a **binary trie** for maximum-XOR queries
- Extend KMP's failure function to **many patterns at once** (Aho-Corasick)
- Count occurrences of every pattern in one pass over the text
- Define the **suffix array** and build it in $O(n \log^2 n)$ / $O(n \log n)$
- Compute the **LCP array** (Kasai) and count distinct substrings

Where we are

- **Week 28:** hashing, prefix function (KMP), Z-function — *one* pattern vs. *one* text
- **Today:** *many* patterns (tries, Aho-Corasick) and *all* substrings (suffix arrays)
- **Later:** suffix array + LCP applications, suffix automaton (Nationals 6), eertree (World Finals 8)

Motivating problem

> Given a text `t` ($|t| \le 10^5$) and k patterns of total length $\le 5 \cdot 10^5$,
> report how many times **each** pattern occurs in `t`.

- KMP per pattern: $O(k \cdot |t| + \Sigma|p_i|)$ → up to $10^5 \cdot k$ — too slow when k is large
- Hashing per distinct length: $O(|t| \cdot \#\text{distinct lengths}) \approx O(|t| \cdot \sqrt{\Sigma})$ — borderline
- **Aho-Corasick:** $O(|t| + \Sigma|p_i| \cdot \sigma)$ — one pass over the text

Tries: the idea

A rooted tree where each edge is labeled by a character; each node = one **prefix** of some word.

```
words: car, cat, cart, dog
        (root)
        /    \
       c      d
       |      |
       a      o
      / \     |
     r*  t*   g*
     |
     t*              (* = end of a word)
```

- Words sharing a prefix share a path → #nodes ≤ $1 + \Sigma|w_i|$

Trie: array implementation

```
const int MAXN = 1'000'005, A = 26;
int nxt[MAXN][A], endCnt[MAXN], passCnt[MAXN], nodes = 1; // node 0 = root

void insert(const string& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (!nxt[v][c]) nxt[v][c] = nodes++;   // 0 means "no child"
        v = nxt[v][c];
        passCnt[v]++;                           // words passing through v
    }
    endCnt[v]++;
}
int countPrefix(const string& p) {              // # words starting with p
    int v = 0;
    for (char ch : p) { v = nxt[v][ch - 'a']; if (!v) return 0; }
    return passCnt[v];
}
```

Trie + DP: Word Combinations (CSES 1731)

> Count ways to split string `s` (|s| ≤ 5000) into words from a dictionary.

- `dp[i]` = number of ways to split the suffix `s[i..]`; `dp[n] = 1`
- From `i`, walk the trie along `s[i], s[i+1], …`
- Every time the walk hits an end-of-word node at depth `j - i`, add `dp[j]`

```
dp[i] = Σ dp[j]   over all j > i with s[i..j) in the dictionary
```

Cost: O(n · maxWordLen) instead of O(n · k · |w|).

Binary trie: maximum XOR

Store numbers as 30-bit strings, **most significant bit first**.

Query `max(x XOR y)` over stored `y`: greedily at each bit, try to go to the child with the **opposite** bit of `x`.

```
x = 0101   stored: 0011, 1000, 1110
bit3: x has 0 → want 1 → go right (1000,1110)
bit2: x has 1 → want 0 → go to 1000
...      → best y = 1000, x^y = 1101
```

- Why greedy works: a higher bit outweighs all lower bits combined
- Keep a **count** per node to support deletions (CF 706D)

From KMP to Aho-Corasick

KMP prefix function: on mismatch, fall back to the **longest proper border**.

Aho-Corasick = KMP on a trie:

- **Failure link** `link[v]`: the node for the **longest proper suffix** of `str(v)` that is also a trie node
- Root and its children: `link = root`
- Compute in **BFS order** (shorter strings first), like π is computed left-to-right

Failure links: example

Patterns: `he, she, his, hers`

node	link
h	root
he	root (no "e" child at root)
s	root
sh	h
she	he
hi	root
his	s
her	root

The goto automaton

Instead of looping over failure links during matching, precompute **transitions** for every (node, char):

```
go(v, c) = child(v, c)            if it exists
         = go(link[v], c)         otherwise   (go(root, c) = root)
```

Computed in BFS order, each in O(1) → whole automaton in **$O(\Sigma|p_i| \cdot \sigma)$**.

Matching then is a single loop: `v = go[v][t[i]]` — $O(|t|)$.

Aho-Corasick: build

```
void build() {
    queue<int> q;
    for (int c = 0; c < A; c++) {
        int u = nxt[0][c];
        if (u == -1) nxt[0][c] = 0;
        else { link[u] = 0; q.push(u); }
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        order.push_back(v);
        for (int c = 0; c < A; c++) {
            int u = nxt[v][c];
            if (u == -1) nxt[v][c] = nxt[link[v]][c];    // goto edge
            else { link[u] = nxt[link[v]][c]; q.push(u); } // trie edge
        }
    }
}
```

Counting all occurrences

Visiting node `v` means **every** pattern on the suffix-link chain of `v` ends here.

Don't walk the chain each time (can be $O(|t|\cdot\sqrt{\Sigma})$ or worse)! Instead:

1. Walk the text: `v = go[v][c]; hit[v]++`
2. Process nodes in **reverse BFS order**: `hit[link[v]] += hit[v]`
3. Answer for pattern i = `hit[end[i]]`

Total: $\mathbf{O(|t| + \Sigma|p_i|\cdot\sigma)}$. The links form a tree (the *fail tree*); step 2 is a subtree sum.

Variants

Question	Technique
Does any pattern occur?	propagate `bad[v] = bad[v] or bad[link[v]]` in BFS order
List all occurrences	"dictionary link" = nearest terminal ancestor in fail tree
Count texts avoiding all patterns	DP over (position, automaton state)
Same with length 10^{18}	matrix power over the automaton (Nationals 7)

The automaton is a DFA: many "string avoids patterns" problems are **DP on automaton states**.

Suffix arrays: definition

`SA` = starting indices of all suffixes of `s`, **sorted lexicographically**.

```
s = banana          i  suffix        sorted:  SA  suffix
                    0  banana                  5  a
                    1  anana                   3  ana
                    2  nana                    1  anana
                    3  ana                     0  banana
                    4  na                      4  na
                    5  a                       2  nana
```

SA = [5, 3, 1, 0, 4, 2]. Every substring = a **prefix of some suffix**.

Building it: prefix doubling

Naive: sort n strings → O(n² log n). Better: sort by first 1, 2, 4, 8, … characters.

- `rank_k[i]` = rank of `s[i..i+k)` among all length-k prefixes
- Key for length 2k: the **pair** `(rank_k[i], rank_k[i+k])` (−1 if past the end)
- Sort by pairs → new ranks; stop when all ranks are distinct

`log n` rounds × sort = **O(n log² n)**; with counting (radix) sort per round → **O(n log n)**.

Suffix array: code (O(n log² n))

```
vector<int> suffixArray(const string& s) {
    int n = s.size();
    vector<int> sa(n), rk(n), tmp(n);
    for (int i = 0; i < n; i++) { sa[i] = i; rk[i] = (unsigned char)s[i]; }
    for (int k = 1;; k <<= 1) {
        auto key = [&](int i) { return i + k < n ? rk[i + k] : -1; };
        auto cmp = [&](int a, int b) {
            return rk[a] != rk[b] ? rk[a] < rk[b] : key(a) < key(b);
        };
        sort(sa.begin(), sa.end(), cmp);
        tmp[sa[0]] = 0;
        for (int i = 1; i < n; i++)
            tmp[sa[i]] = tmp[sa[i - 1]] + cmp(sa[i - 1], sa[i]);
        rk = tmp;
        if (rk[sa[n - 1]] == n - 1) break;   // all distinct
    }
    return sa;
}
```

The LCP array

`lcp[i]` = longest common prefix of suffixes `SA[i]` and `SA[i+1]`.

```
SA  suffix   lcp with next
5   a        1
3   ana      3
1   anana    0
0   banana   0
4   na       2
2   nana     -
```

Kasai's algorithm, O(n): process suffixes in text order `i = 0, 1, …`.

Key fact: if suffix `i` has LCP `h` with its predecessor, suffix `i+1` has LCP ≥ `h − 1`.

Kasai: code

```
vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size(), h = 0;
    vector<int> rk(n), lcp(max(n - 1, 0));
    for (int i = 0; i < n; i++) rk[sa[i]] = i;
    for (int i = 0; i < n; i++) {
        if (rk[i] == 0) { h = 0; continue; }
        int j = sa[rk[i] - 1];                  // previous suffix in SA
        while (i + h < n && j + h < n && s[i + h] == s[j + h]) h++;
        lcp[rk[i] - 1] = h;
        if (h > 0) h--;
    }
    return lcp;
}
```

`h` decreases at most n times and never exceeds n → O(n) total.

Applications (first taste)

- **Pattern search:** binary search for `p` in the SA → O(|p| log n); all occurrences form a contiguous SA range
- **Distinct substrings:** each suffix `SA[i]` contributes `n − SA[i] − lcp[i−1]` new ones
  → total = n(n+1)/2 − Σ lcp (banana: 21 − 6 = 15)
- **Longest repeated substring:** max of lcp
- **Longest common substring of A, B:** SA of `A#B`, adjacent pairs from different strings

Full treatment (LCP + sparse table, k-th substring, etc.): **Nationals 6**.

Complexity summary

Structure	Build	Query
Trie	$O(\Sigma\|w_i\|)$ time, $O(\Sigma\|w_i\|\cdot\sigma)$ memory	$O(\|p\|)$
Binary trie	$O(n\cdot B)$	$O(B)$ per max-XOR
Aho-Corasick	$O(\Sigma\|p_i\|\cdot\sigma)$	$O(\|t\|)$ + $O(\#\text{nodes})$ to aggregate
Suffix array	$O(n \log^2 n)$ or $O(n \log n)$	$O(\|p\| \log n)$ search
LCP (Kasai)	$O(n)$	—

Common pitfalls

- Trie arrays sized by #words instead of **total length + 1** → RE
- `int nxt[N][26]` for N = 10^6 is 104 MB — check the memory limit
- Aho-Corasick: computing links in **DFS** order (wrong — needs BFS)
- Walking the whole suffix-link chain for every text position → TLE
- Duplicate patterns: store `end[i]` per pattern, not a single id per node
- Suffix array: forgetting that `rk[i+k]` may be out of range
- Mixing up `lcp[i]` (between SA[i], SA[i+1]) and `lcp[i]` (between SA[i−1], SA[i])

Practice problems

- **CSES 1731** Word Combinations — trie + DP
- **CF 706D** Vasiliy's Multiset — binary trie with deletions
- **CF 923C** Perfect Security — binary trie, greedy min-XOR
- **CSES 2102 / 2103 / 2104** Finding / Counting Patterns, Pattern Positions — Aho-Corasick or SA
- **Kattis stringmultimatching** — Aho-Corasick, all positions
- **CSES 2105** Distinct Substrings — SA + LCP
- **CSES 2106** Repeating Substring — max LCP
- **Kattis suffixsorting** — build an SA

Summary

- **Trie** = shared prefixes; O(|s|) insert/lookup; binary trie for XOR problems
- **Aho-Corasick** = trie + failure links (KMP generalized), built by BFS
- Aggregate matches via the **fail tree**, not by chain-walking
- **Suffix array** = sorted suffixes; prefix doubling builds it
- **LCP** via Kasai in O(n); unlocks distinct substrings, repeats, LCS

Next week: combinatorics, inclusion-exclusion, probability & expected value.